Топ-30 возможностей Quarkus

Oct 24, 2026 · @Anat

Quarkus переносит максимум работы фреймворка с запуска на сборку, и из этого вытекает большая часть его преимуществ: быстрый старт, малая память и хорошая совместимость с нативной компиляцией. Ниже тридцать возможностей, ради которых его стоит рассматривать, по 3-5 технических аргументов к каждой.

Группа 1. Архитектура и производительность

1. Оптимизация на этапе сборки

- Сканирование classpath, разбор аннотаций и построение метаданных выполняются при сборке, а не при старте. Рантайм получает уже готовый результат.
- Отсюда меньше рефлексии и динамических прокси в горячем пути, а значит меньше работы для JIT и меньше шансов на неожиданное поведение.
- Расширения (extensions) делятся на runtime и deployment части, поэтому тяжёлый код обработки не попадает в боевой артефакт.
- Вывод: ошибки конфигурации и внедрения зависимостей часто ловятся на сборке, а не на проде.

> **Примеры к пункту 1**
>
> - К аргументу про перенос работы на сборку: расширение описывает обработку через build step, который выполняется при сборке и не попадает в рантайм.
>
> ```
> @BuildStep
> void registerBeans(BuildProducer<AdditionalBeanBuildItem> beans) {
>
> beans.produce(AdditionalBeanBuildItem.unremovableOf(MyService.class));
> }
> ```
>
> - К аргументу про ошибки на сборке: если у бина нет подходящей зависимости, сборка падает с ошибкой вроде UnsatisfiedResolutionException, а не приложение при старте на проде.
> - К аргументу про меньше рефлексии: вместо сканирования во время запуска на выходе получаются уже сгенерированные классы. Замеры из источников ниже показывают итоговый эффект на старте.
> - Цифры: в сравнении Java Code Geeks (декабрь 2025) нативный Quarkus стартует за 0,049 с, нативный Spring Boot за 0,104 с. Проверено по самой странице: цифры совпадают. Оговорки: это сравнение стороннего автора, а по подписям к графикам данные взяты из отдельного бенчмарка Java 25, так что это ориентир, не гарантия. Источник

2. Нативная компиляция через GraalVM и Mandrel

- Старт измеряется десятками миллисекунд, что делает осмысленным масштабирование до нуля в serverless.
- Заметно ниже потребление памяти (RSS) по сравнению с обычной JVM, что позволяет плотнее упаковывать поды.
- Сборку можно выполнять внутри контейнера, поэтому локально GraalVM устанавливать не обязательно.
- Платой служат более долгая сборка и ограничения на динамику (рефлексия, ресурсы), которые Quarkus во многом закрывает через аннотации и автоматическую регистрацию.
- Для пиковой пропускной способности JVM-режим с JIT нередко остаётся лучшим выбором, поэтому режим стоит подбирать под профиль нагрузки.

Примеры к пункту 2

- К аргументу про старт: в замере ForTek нативный образ стартовал за 0,079 секунды против 2,292 секунды в JVM-режиме, то есть примерно в 29 раз быстрее. Проверено по странице: цифры совпадают. Оговорки: это тест одного автора на Quarkus 3.15.1, в выводе docker stats у JVM-контейнера виден лимит памяти 128 МиБ, а сам автор пишет, что после прогрева JVM может обгонять нативный режим. Источник
- К аргументу про память: в том же замере память при старте была 16,27 МиБ у нативного образа против 113,5 МиБ у JVM, то есть на 86% меньше. Источник
- Сравнение по RSS в статье HackerNoon: около 70 МБ у нативного Quarkus против 149 МБ у Spring Boot, а куча 3 МБ против 11 МБ. Проверено по странице. В сравнении Java Code Geeks те же числа приведены точнее: 70,5 и 149,4 МБ. Это пересказ одного и того же замера, а не независимые подтверждения. Источник
- К аргументу про сборку в контейнере:

```
./mvnw package -Dnative -Dquarkus.native.container-build=true
```

- К аргументу про пиковую пропускную способность: в блоге Quarkus сказано, что JVM-режим даёт лучшую пропускную способность и время отклика, чем нативный, но потребляет до 277% больше памяти (RSS). Проверено: фраза на странице есть, но запись очень старая, и там сравнение идёт с Thorntail, так что как свежую цифру её брать не стоит. Источник Более новые данные есть в руководстве по нативной сборке: в таблице режимов память у JVM fast-jar 304 МиБ, у JVM с Leyden AOT 240 МиБ, у нативного образа 95 МиБ, а пропускная способность нативного образа примерно на 59% ниже, чем у JVM (у Leyden примерно на 7% ниже). Условия замера смотрите на самой странице. Блог Quarkus о новых бенчмарках говорит похожее: в нативном режиме пропускная способность примерно вдвое ниже.

3. Единая реактивная и императивная модель

- Под капотом лежит Vert.x с неблокирующим циклом событий, но писать можно в привычном императивном стиле.
- Аннотации вроде Blocking и NonBlocking явно управляют тем, в каком потоке выполняется метод.

- Не нужно переписывать приложение целиком: реактивные части подключаются точечно там, где это даёт выигрыш.
- Модель работает и с виртуальными потоками, хотя на Java 25 были зафиксированы случаи блокировки (pinning) при загрузке классов, поэтому такие сценарии нужно тестировать отдельно.

> **Примеры к пункту 3**
>
> - К аргументу про императивный стиль поверх Vert.x: обычный метод возвращает значение, а Quarkus сам решает, в каком потоке его выполнить.

```
@GET
@Path("/plain")
public String plain() {
    return "синхронный стиль";
}
```

> - К аргументу про явное управление потоком:

```
@GET
@Path("/reactive")
@NonBlocking
public Uni<String> reactive() {
    return Uni.createFrom().item("реактивный стиль");
}

@GET
@Path("/slow")
@Blocking
public String slow() {
    return heavyBlockingCall();
}
```

> - К аргументу про виртуальные потоки: метод помечается аннотацией RunOnVirtualThread.

```
@GET
@RunOnVirtualThread
public String viaVirtualThread() {
    return blockingRepository.load();
}
```

> Про блокировки при загрузке классов на Java 25 см. обсуждение.

4. ArC: внедрение зависимостей на этапе сборки

- Реализация CDI Lite, где граф бинов строится при сборке, без рантайм-рефлексии.
- Неиспользуемые бины удаляются из итогового артефакта, что уменьшает размер и ускоряет старт.
- Ошибки вроде неразрешённых или неоднозначных зависимостей обнаруживаются при сборке.
- Цена: часть динамических возможностей полного CDI не поддерживается, и это стоит знать заранее при миграции.

> **Примеры к пункту 4**
>
> - К аргументу про CDI без рефлексии:
>
> ```
> @ApplicationScoped
> public class PriceService {
>     public double price(String sku) { return 42.0; }
> }
>
> @Path("/price")
> public class PriceResource {
>     @Inject
>     PriceService service;
> }
> ```
>
> - К аргументу про удаление неиспользуемых бинов: бин, на который нигде нет ссылки, будет убран при сборке. Чтобы сохранить такой бин, используют аннотацию Unremovable.
>
> ```
> @Unremovable
> @ApplicationScoped
> public class LookedUpByNameBean { }
> ```
>
> - К аргументу про ошибки на сборке: если добавить второй бин того же типа без квалификатора, сборка завершится ошибкой неоднозначной зависимости.

5. Поддержка Java 25 и свежего стека

- Полная поддержка Java 25 появилась в версии 3.31, а актуальная ветка сейчас 3.40.

- В одном релизе обновились Hibernate ORM до 7.2, JUnit до 6 и Testcontainers до 2, то есть стек не отстаёт от экосистемы.
- Выпускаются образы на базе UBI9 с OpenJDK 25, что упрощает путь в корпоративные окружения.
- Для проверки совместимости есть отдельная рабочая группа по Java 25 в репозитории проекта.

> **Примеры к пункту 5**
>
> - К аргументу про версии: в файле pom.xml версия компилятора задаётся так.
>
> ```
> <properties>
>     <maven.compiler.release>25</maven.compiler.release>
>     <quarkus.platform.version>3.31.0</quarkus.platform.version>
> </properties>
> ```
>
> Конкретный номер версии подставьте актуальный с quarkus.io.
>
> - К аргументу про обновление стека: в одном релизе 3.31 Hibernate ORM обновился до 7.2, Hibernate Reactive до 3.2, а также JUnit до 6 и Testcontainers до 2. Источник
> - К аргументу про образы: опубликованы образы registry.access.redhat.com/ubi9/openjdk-25 и registry.access.redhat.com/ubi9/openjdk-25-runtime. Источник

Группа 2. Опыт разработчика

6. Режим разработки с живой перезагрузкой

- Команда quarkus dev отслеживает изменения кода, ресурсов и конфигурации и применяет их при следующем запросе, без ручного перезапуска.
- Цикл правка-проверка сокращается до секунд, что заметно при работе с тяжёлыми приложениями.
- Перезагрузка работает не только для классов, но и для конфигурации и многих расширений.
- Можно подключить удалённый режим разработки для работы с приложением в контейнере или кластере.

> **Примеры к пункту 6**
>
> - К аргументу про запуск режима разработки:

```
./mvnw quarkus:dev
```

> - К аргументу про цикл правка-проверка: изменения применяются при следующем обращении, например при вызове REST-метода или обновлении страницы, а ручной перезапуск не нужен. Источник
> - К аргументу про удалённый режим: для работы с приложением в контейнере или кластере включается remote dev, при этом в боевой сборке нужно задать секрет.

```
quarkus.package.jar.type=mutable-jar
quarkus.live-reload.password=changeit
quarkus.live-reload.url=http://my-cluster-host:8080
```

> Проверено по материалам Red Hat, Baeldung и Себастьяна Дашнера: для удалённой разработки приложение пакуется как mutable-jar, в локальных настройках задаются свойства quarkus.live-reload.url и quarkus.live-reload.password, а подключение запускается командой quarkus:remote-dev (Red Hat Developer, Baeldung). Важная поправка: в старых статьях тип пакета задаётся свойством quarkus.package.type, а в новых версиях используется quarkus.package.jar.type, как в Baeldung и в материале LogicMonitor. Для вашей версии 3.x берите вариант с jar.type и сверьте его с руководством по Maven.

7. Dev Services

- Если не указан адрес базы, брокера или другого сервиса, Quarkus сам поднимает его в контейнере на время разработки и тестов.
- Новому разработчику не нужно вручную настраивать окружение: достаточно запустить проект.
- Поддерживаются базы данных, Kafka, Redis, брокеры AMQP, OIDC-провайдеры и другие сервисы.
- Конфигурация подставляется автоматически, а в боевом профиле используются ваши настоящие адреса.

> **Примеры к пункту 7**
>
> - К аргументу про автоматический запуск сервисов: при адресе базы, не заданном в конфигурации, в логах появляется запись о том, что Dev Services для источника данных по умолчанию запустили PostgreSQL в контейнере. Источник
> - К аргументу про отсутствие ручной настройки: достаточно подключить расширение драйвера и не указывать адрес.
>
> ```
> <dependency>
>     <groupId>io.quarkus</groupId>
>     <artifactId>quarkus-jdbc-postgresql</artifactId>
> </dependency>
> ```
>
> - К аргументу про остановку: при остановке режима разработки по Ctrl+C контейнер базы тоже останавливается, это можно проверить командой docker ps. Источник
> - К аргументу про боевой профиль: как только вы задаёте настоящий адрес для профиля prod, Dev Services на нём не используются.
>
> ```
> %prod.quarkus.datasource.jdbc.url=jdbc:postgresql://db:5432/app
> ```

8. Dev UI

- Веб-интерфейс по адресу /q/dev-ui показывает бины, конфигурацию и данные расширений во время работы приложения.
- Расширения добавляют собственные карточки, например метрики Hibernate ORM или просмотр бинов ArC.
- Часть действий выполняется прямо из интерфейса через JSON-RPC.
- Интерфейс запоминает настройки и историю команд для каждого проекта отдельно.

> **Примеры к пункту 8**
>
> - К аргументу про адрес интерфейса: Dev UI доступен по адресу localhost:8080/q/dev-ui, когда приложение запущено в режиме разработки. Источник
> - К аргументу про карточки расширений: каждое расширение добавляет свою карточку, например просмотр бинов ArC или метрики Hibernate ORM.
> - К аргументу про обмен данными: интерфейс общается с бэкендом через JSON-RPC, а расширения регистрируют свои страницы при сборке. Источник
> - К аргументу про запуск в режиме разработки: Dev UI включается вместе с Dev Services, горячей перезагрузкой и непрерывным тестированием, а также открывает порт отладки 5005. Источник

9. Непрерывное тестирование

- Прямо в режиме разработки тесты запускаются автоматически при изменении кода, а выполняются только затронутые тесты.
- Результаты видны в терминале сразу, без отдельного запуска сборки.
- Вместе с Dev Services это даёт полноценные интеграционные тесты с реальной базой при минимуме настроек.

> **Примеры к пункту 9**
>
> - К аргументу про запуск тестов в фоне: непрерывное тестирование можно запустить отдельной командой.
>
> ```
> ./mvnw quarkus:test
> ```
>
> Источник
>
> - К аргументу про выбор тестов: набор задаётся свойствами с шаблонами включения и исключения. Источник
>
> ```
> quarkus.test.include-pattern=.*IT
> quarkus.test.exclude-pattern=.*Slow.*
> ```
>
> - К аргументу про автозапуск: чтобы тесты шли сразу при старте без нажатия клавиши, включают свойство.
>
> ```
> quarkus.test.continuous-testing=enabled
> ```
>
> Источник
>
> - К аргументу про скорость: в примере из статьи на Medium непрерывное тестирование сообщает, что один тест выполнен за 1547 миллисекунд. Проверено: тот же вывод есть и в статье на Хабре. Это единичный пример на Quarkus 2.0 (2021 год), а не эталонный замер скорости. Источник

10. Единая конфигурация с профилями

- Источники конфигурации на основе MicroProfile Config объединяются: файл свойств, переменные окружения, системные свойства.
- Профили задаются префиксом, например profile prod или profile test, что позволяет держать всё в одном файле.
- Значения можно типизировать через интерфейсы с аннотацией ConfigMapping.
- Поддерживается подстановка выражений и значений по умолчанию.

Примеры к пункту 10

- К аргументу про объединение источников: значение можно задать в файле, переменной окружения или системном свойстве, и они складываются по приоритету.

```
greeting.message=Привет
```

```
GREETING_MESSAGE=Hello java -jar target/quarkus-app/quarkus-run.jar
```

- К аргументу про профили: префикс процента ограничивает значение профилем.

```
%dev.quarkus.http.port=9090
%test.quarkus.http.port=0
%prod.quarkus.http.port=8080
```

- К аргументу про типизацию: настройки собираются в интерфейс.

```
@ConfigMapping(prefix = "greeting")
public interface GreetingConfig {
    String message();

    @WithDefault("Anat")
    String name();
}
```

- К аргументу про подстановки: в значении можно ссылаться на другое свойство.

```
app.url=http://${app.host:localhost}:${quarkus.http.port}
```

Группа 3. Данные и персистентность

11. Hibernate ORM с Panache

- Два стиля на выбор: Active Record, где сущность сама умеет искать и сохранять себя, и классический репозиторий.
- Сокращённые запросы без полного HQL убирают шаблонный код и делают простые случаи читаемыми.
- Сущности расширяют PanacheEntity, который уже содержит идентификатор, что убирает рутину.

- Под низом остаётся обычный Hibernate, поэтому при необходимости доступны все его возможности.

> **Примеры к пункту 11**
>
> - К аргументу про два стиля: Panache поддерживает паттерны Active Record и Repository, и оба дают стандартные методы сохранения, чтения, обновления и удаления. Источник
> - К аргументу про Active Record: сущность наследует PanacheEntity.

```
@Entity
public class Customer extends PanacheEntity {
    public String name;

    public static List<Customer> findByName(String name) {
        return list("name", name);
    }
}
```

> - К аргументу про репозиторий: логика выносится в отдельный класс.

```
@ApplicationScoped
public class MovieRepository implements PanacheRepository<Movie> {
    public List<Movie> findByYear(int year) {
        return find("YEAR(releaseDate)", year).list();
    }
}
```

> Источник примера
>
> - К аргументу про запросы рядом с сущностью: в руководстве советуют держать запросы внутри самих сущностей, чтобы их было легко найти. Источник
> - К аргументу про доступ ко всем возможностям Hibernate: подключение расширений делается командой.

```
mvn quarkus:add-extension -Dextensions="hibernate-orm-panache, jdbc-
postgresql"
```

12. Hibernate Reactive

- Неблокирующий доступ к базе поверх Vert.x SQL clients для приложений на реактивной модели.
- Версия 3.2 поставляется вместе с Hibernate ORM 7.2 в релизе 3.31.

- Поддерживаются несколько реактивных фабрик сессий для разных источников данных.
- Стоит выбирать там, где основная нагрузка связана с ожиданием ввода-вывода, и оценивать на реальном профиле.

> **Примеры к пункту 12**
>
> - К аргументу про реактивный доступ: в обзорной статье Djamware рекомендуют держать реактивные и императивные источники данных раздельно. Источник
> - К аргументу про несколько источников: именованные источники задаются в конфигурации.
>
> ```
> quarkus.datasource.users.db-kind=postgresql
> quarkus.datasource.products.db-kind=postgresql
> quarkus.hibernate-orm.users.datasource=users
> quarkus.hibernate-orm.products.datasource=products
> ```
>
> Пример адаптирован из обзора The Main Thread.
>
> - К аргументу про версии: Hibernate Reactive 3.2 вышел вместе с Hibernate ORM 7.2 в релизе 3.31. Источник
> - Пример реактивного API: по руководству по Hibernate Reactive и материалу Mastertheboss фабрика сессий внедряется как Mutiny.SessionFactory, а запрос выполняется внутри withTransaction и возвращает Uni, например sf.withTransaction((s, t) -> s.find(Customer.class, id)). Важное замечание из руководства по миграции на 3.0: внедрять SessionFactory рекомендуется вместо внедрения самой сессии, потому что жизненный цикл реактивной сессии не совпадает с контекстом запроса CDI. И ещё один свежий нюанс: в issue 57396 описано, что начиная с версии 3.38 аннотации WithSession и WithTransaction открывают сессию в момент вызова метода, а не подписки на Uni. Это открытая проблема, так что для актуального поведения смотрите её статус.

13. Panache Next и Jakarta Data

- Новое поколение Panache, появившееся в релизе 3.31.
- Развивается вместе со спецификацией Jakarta Data, что даёт стандартизированный подход к репозиториям.
- Безопасность поддерживается и на уровне репозиториев Jakarta Data через аннотации.

- Оговорка: проект активно меняется. Источники расходятся в названии, в одних он называется Panache Next, в других Quarkus Data, поэтому пакеты и имена расширений перед использованием сверьте с текущим блогом Quarkus.

> **Примеры к пункту 13**
>
> - К аргументу про новое поколение Panache: оно вошло в релиз 3.31 одновременно с обновлением Hibernate ORM до 7.2. Источник
> - К аргументу про безопасность репозиториев: в релизе 3.31 добавлена поддержка аннотаций безопасности на репозиториях Jakarta Data. Источник
> - Про Panache Next: по блогу Quarkus репозиторий реализует PanacheRepository.Reactive.Stateless для реактивной работы без состояния, запросы возвращают Uni, и можно сочетать режимы для разных сущностей. По разбору The Main Thread пакет у него другой, чем у классического Panache: io.quarkus.hibernate.panache вместо io.quarkus.hibernate.orm.panache. Но тот же автор позже пишет, что проект переименовали в Quarkus Data, и отправной точкой стало расширение quarkus-data-hibernate. Поскольку название и пакеты меняются, готовый код я не приводила: актуальный пример берите из текущего руководства на quarkus.io.
> - Для сравнения, стабильный стиль репозитория выглядит так.
>
> ```
> @ApplicationScoped
> public class BookRepository implements PanacheRepository<Book> {
>     public long countOlderThan(int year) {
>         return count("year < ?1", year);
>     }
> }
> ```

14. Несколько источников данных и единиц персистентности

- Каждому источнику даётся имя в конфигурации, например отдельные настройки для пользователей и товаров.
- Единицы персистентности Hibernate привязываются к конкретным источникам данных.
- Это упрощает работу с несколькими базами или схемами в одном сервисе.
- Раздельная конфигурация остаётся декларативной и не требует ручной сборки фабрик.

> **Примеры к пункту 14**
>
> - К аргументу про именованные источники: у каждого источника своё имя в ключе конфигурации.

```
quarkus.datasource.users.db-kind=postgresql
quarkus.datasource.users.jdbc.url=jdbc:postgresql://users-db:5432/users
quarkus.datasource.products.db-kind=postgresql
quarkus.datasource.products.jdbc.url=jdbc:postgresql://products-
db:5432/products
```

> - К аргументу про привязку единиц персистентности: каждая единица указывает свой источник.

```
quarkus.hibernate-orm.users.datasource=users
quarkus.hibernate-orm.users.packages=org.acme.users
quarkus.hibernate-orm.products.datasource=products
quarkus.hibernate-orm.products.packages=org.acme.products
```

> - К аргументу про отдельные пакеты сущностей: сущности привязываются к единице через пакеты, как в примере выше.
> - Проверено по руководству по Hibernate ORM: свойства в корне пространства quarkus.hibernate-orm задают единицу по умолчанию, а именованные единицы задаются через имя в пространстве. Из исходников расширения видно, что у единицы есть свойства datasource и packages. Примеры из обсуждений подтверждают форму quarkus.hibernate-orm."имя".datasource и quarkus.hibernate-orm."имя".packages. Практическое замечание: при нескольких источниках данных нужно либо настроить источник по умолчанию, либо явно привязать все единицы, иначе возникает ошибка про ненайденный источник, как в issue 44807.

15. Миграции схемы: Flyway и Liquibase

- Расширения автоматически выполняют миграции при старте приложения.
- Совместно с Dev Services вы получаете чистую базу с актуальной схемой в каждом запуске разработки или тестов.
- Для боевых окружений миграции можно запускать отдельным шагом, не смешивая их со стартом сервиса.
- Это снимает необходимость в стратегии автосоздания схемы Hibernate за пределами разработки.

> **Примеры к пункту 15**
>
> - К аргументу про расширения: Quarkus предоставляет расширения и для Flyway, и для Liquibase, и автор руководства рекомендует их вместо автогенерации схемы. Источник
> - К аргументу про файлы миграций: скрипты кладутся в каталог ресурсов по умолчанию.
>
> ```
> -- src/main/resources/db/migration/V1__create_books.sql
> CREATE TABLE books (
>     id SERIAL PRIMARY KEY,
>     title VARCHAR(255) NOT NULL,
>     author VARCHAR(255),
>     year INT
> );
> ```
>
> Источник примера
>
> - К аргументу про запуск при старте:
>
> ```
> quarkus.flyway.migrate-at-start=true
> ```
>
> - К аргументу про боевые окружения: схему не создаёт Hibernate, а отключается автогенерация.
>
> ```
> %prod.quarkus.hibernate-orm.database.generation=none
> ```
>
> Совет не использовать значение update в боевой среде приведён в том же обзоре. Источник

Группа 4. Сетевой слой, REST и реактивность

16. Quarkus REST

- Реализация Jakarta REST, построенная на Vert.x и спроектированная под реактивную модель с самого начала.
- Выбор потока выполнения определяется по сигнатуре метода: возвращаемые реактивные типы идут в поток цикла событий, остальные могут уходить в рабочий пул.
- Обработка запросов во многом готовится при сборке, поэтому накладные расходы на рефлексию минимальны.
- Доступна поддержка Jackson и JSON-B для сериализации.

> **Примеры к пункту 16**
>
> - К аргументу про реактивную модель с самого начала: метод возвращает Uni со списком, а Quarkus REST сам превращает его в JSON-массив. Источник

```
@GET
public Uni<List<Fruit>> list() {
    return Fruit.listAll();
}
```

> - К аргументу про выбор потока по сигнатуре: возвращаемый реактивный тип идёт в поток цикла событий, а обычный тип можно пометить явно.

```
@GET
@Path("/{id}")
public Uni<Fruit> get(Long id) {
    return Fruit.findById(id);
}
```

> - К аргументу про транзакции в реактивном стиле: создание записи оборачивается в реактивную транзакцию. Источник

```
@POST
public Uni<RestResponse<Fruit>> create(Fruit fruit) {
    return Panache.withTransaction(fruit::persist)
        .replaceWith(RestResponse.status(CREATED, fruit));
}
```

> - К аргументу про сериализацию: JSON подключается расширением rest-jackson, как в команде создания проекта выше.

17. Реактивный клиент REST

- Декларативные клиенты описываются интерфейсом с аннотациями, а реализация генерируется при сборке.
- Поддерживаются как блокирующие, так и реактивные возвращаемые типы.
- Клиент интегрируется с отказоустойчивостью, трассировкой и безопасностью без ручной обвязки.
- Настройка адресов и таймаутов выносится в конфигурацию.

Примеры к пункту 17

- К аргументу про декларативный клиент: интерфейс с аннотациями, реализация генерируется при сборке. Реактивные методы возвращают Uni. Источник

```
@RegisterRestClient(configKey = "world-clock")
public interface WorldClockService {
    @GET
    @Path("/json/cet/now")
    @Produces(MediaType.APPLICATION_JSON)
    Uni<WorldClock> getNow();
}
```

- К аргументу про блокирующие и реактивные типы: тот же интерфейс может возвращать обычное значение вместо Uni, это решает сигнатура метода.
- К аргументу про настройку адресов и таймаутов: всё выносится в конфигурацию по ключу клиента.

```
quarkus.rest-client.world-clock.url=http://worldclockapi.com/api
quarkus.rest-client.world-clock.read-timeout=3000
```

Проверено по материалам: префикс quarkus.rest-client с ключом клиента и свойством url подтверждается примерами (DEV Community, Baeldung). Свойство read-timeout существует, но в обсуждении на GitHub отмечено, что оно имеет числовой тип, а не Duration, так что значение 3000 читается как миллисекунды (issue 22872). Это обсуждение старое, поэтому единицы всё же сверьте со страницей Using the REST Client.

- К аргументу про использование в ресурсе: клиент внедряется аннотацией RestClient. Источник примера

```
@RestClient
WorldClockService clock;
```

18. Smallrye Mutiny

- Реактивная библиотека с двумя основными типами: Uni для одного результата и Multi для потока.
- API построен вокруг читаемых цепочек с явными методами для обработки событий, ошибок и завершения.
- Единый подход используется в клиентах баз данных, сообщениях и REST.

- Преобразование между Uni и CompletionStage облегчает интеграцию с другим кодом.

> **Примеры к пункту 18**
>
> - К аргументу про два типа: Uni отдаёт один результат, Multi отдаёт поток. Источник
> - К аргументу про цепочки преобразований: поток слов приводится к верхнему регистру, и Quarkus отдаёт его как JSON-поток. Источник
>
> ```
> @GET
> @Produces(MediaType.APPLICATION_JSON)
> public Multi<String> words() {
>     return Multi.createFrom().items("one", "two", "three")
>         .onItem().transform(String::toUpperCase);
> }
> ```
>
> - К аргументу про комбинирование Multi и Uni: каждый элемент потока можно преобразовать в отдельный Uni.
>
> ```
> Multi.createFrom().items("a", "b", "c")
>     .onItem().transformToUni(s -> Uni.createFrom().item(s + "_x"))
>     .concatenate();
> ```
>
> Источник
>
> - К аргументу про безопасную обёртку блокирующего кода: блокирующий вызов запускается на виртуальных потоках. Источник
>
> ```
> Uni<String> blockingWrapped = Uni.createFrom()
>     .item(() -> blockingCall())
>     .runSubscriptionOn(Executors.newVirtualThreadPerTaskExecutor());
> ```

19. WebSockets и Server-Sent Events

- Поддержка двусторонней связи через WebSockets Next с декларативной моделью обработчиков.
- Server-Sent Events доступны через возвращаемый тип Multi, что даёт потоковую передачу данных клиенту без лишнего кода.
- Оба варианта работают на неблокирующем движке Vert.x и подходят для большого числа соединений.

Примеры к пункту 19

- К аргументу про Server-Sent Events через Multi: метод возвращает поток, а клиент получает данные по мере появления. Источник

```
@GET
@Path("/stream")
@Produces(MediaType.SERVER_SENT_EVENTS)
public Multi<String> stream() {
    return Multi.createFrom().ticks().every(Duration.ofSeconds(1))
        .map(n -> "tick " + n);
}
```

- К аргументу про большое число соединений: автор материала про gRPC и Mutiny отмечает, что тот же код масштабируется на тысячи одновременных соединений. Это его оценка, а не замер. Источник
- К аргументу про WebSockets Next: пример ниже построен по руководству по WebSockets Next и справочнику. Конечная точка объявляется аннотацией WebSocket с путём, соединение внедряется как WebSocketConnection, а обработчики помечаются аннотациями OnOpen, OnClose и OnTextMessage. Параметр broadcast рассылает результат всем подключённым.

```
@WebSocket(path = "/chat/{username}")
public class ChatWebSocket {

    public record ChatMessage(String from, String message) { }

    @Inject
    WebSocketConnection connection;

    @OnOpen(broadcast = true)
    public ChatMessage onOpen() {
        return new ChatMessage(connection.pathParam("username"),
"joined");
    }

    @OnTextMessage(broadcast = true)
    public ChatMessage onMessage(ChatMessage message) {
        return message;
    }
}
```

> Это сокращённая версия примера из руководства: я убрала перечисление типов сообщений и обработчик закрытия. Импорты берутся из пакета io.quarkus.websockets.next.

20. gRPC

- Расширение генерирует код сервера и клиента по описанию protobuf при сборке.
- Поддерживаются все четыре режима вызова, включая потоковые, с использованием Mutiny.
- Сервисы gRPC могут сосуществовать с REST в одном приложении и на одном порту.
- Интеграция с проверкой состояния и метриками работает из коробки.

> **Примеры к пункту 20**
>
> - К аргументу про генерацию кода по protobuf: файл описания кладётся в каталог src/main/proto, а классы генерируются при сборке. Источник
> - К аргументу про зависимости: подключается расширение gRPC.

```
<dependency>
    <groupId>io.quarkus</groupId>
    <artifactId>quarkus-grpc</artifactId>
</dependency>
```

> - К аргументу про сосуществование с REST: gRPC-клиент внедряется в REST-ресурс аннотацией GrpcClient. Источник

```
@GrpcClient("filesystem")
FileManager service;
```

> - К аргументу про потоковые режимы и Mutiny: в разборе The Main Thread потоковая передача строится на Multi, при закрытии канала клиентом приходит ошибка UNAVAILABLE, это сигнал о штатном закрытии, а не сбой. Источник
> - Пример из источника написан для старой версии, где пространство имён было javax, а не jakarta. В новых версиях замените импорты.

Группа 5. Сообщения и интеграции

21. Reactive Messaging

- Единая абстракция над Kafka, AMQP, MQTT и другими брокерами с каналами, входящими и исходящими.
- Обработчики описываются аннотациями Incoming и Outgoing, а подключение к конкретному брокеру меняется конфигурацией.
- Поддерживается управление подтверждением сообщений и обратным давлением.
- Удобно менять брокер или подменять его в тестах без изменения бизнес-кода.

> **Примеры к пункту 21**
>
> - К аргументу про каналы: для каждой записи из входящего топика вызывается метод обработки, а результат уходит в исходящий канал. Источник

```
@Incoming("requests")
@Outgoing("quotes")
public Quote process(String quoteRequest) {
    return new Quote(quoteRequest, random.nextInt(100));
}
```

> - К аргументу про смену брокера конфигурацией: имя канала в аннотации связывается с настройками по шаблону.

```
mp.messaging.incoming.requests.topic=quote-requests
mp.messaging.incoming.requests.auto.offset.reset=earliest
```

> Шаблон ключей описан в руководстве: mp.messaging.[outgoing|incoming].{имя-канала}.свойство.
>
> - К аргументу про подтверждение сообщений: если принимать Message, подтверждать успех и отказ нужно вручную через ack и nack. Источник
> - К аргументу про тесты: если канал не настроен, по умолчанию используется канал в памяти, и это удобно для тестирования. Источник

22. Kafka и Kafka Streams

- Расширение для Kafka использует тот же реактивный API, а сериализаторы настраиваются через конфигурацию.

- Dev Services поднимают брокер автоматически, что сильно упрощает локальную разработку.
- Поддерживается интеграция с реестром схем.
- Для потоковой обработки доступно отдельное расширение Kafka Streams.

> **Примеры к пункту 22**
>
> - К аргументу про тот же реактивный API: Quarkus работает с Kafka через SmallRye Reactive Messaging. Источник
> - К аргументу про настройку сериализаторов: расширение часто само определяет сериализатор и десериализатор по аннотациям Incoming, Outgoing и по внедрённым каналам. Источник
>
> ```
> <dependency>
>     <groupId>io.quarkus</groupId>
>     <artifactId>quarkus-smallrye-reactive-messaging-kafka</artifactId>
> </dependency>
> ```
>
> - К аргументу про начало чтения с первой записи:
>
> ```
> mp.messaging.incoming.requests.auto.offset.reset=earliest
> ```
>
> Значение earliest означает, что приложение читает с первого смещения, когда для группы потребителей нет сохранённого. Источник
>
> - К аргументу про Kafka Streams и Dev Services: по руководству по Kafka Streams нужно добавить расширение quarkus-kafka-streams и объявить в CDI-бине метод с аннотацией Produces, возвращающий Topology, собранный через StreamsBuilder. Это же подтверждает заметка Piotr's TechBlog. Свойства вида quarkus.kafka-streams.bootstrap-servers и quarkus.kafka-streams.application-id встречаются в материале Hatim Tachi. Топологию можно посмотреть в Dev UI, как описано в статье Red Hat. Про Dev Services именно для Kafka Streams подтверждения не нашла.

23. Redis, кэш и клиенты данных

- Клиент Redis доступен в реактивном и императивном вариантах.
- Расширение кэша позволяет кэшировать результаты методов аннотациями, например CacheResult.
- Dev Services запускают Redis в контейнере для разработки.

- Оговорка: в релизе 3.40.1 исправлялась ошибка, при которой CacheResult с Redis под высокой параллельной нагрузкой возвращал значение QUEUED, поэтому при обновлениях стоит держаться свежих патчей.

> **Примеры к пункту 23**
>
> - К аргументу про кэш аннотациями: метод помечается CacheResult, и результат кэшируется по ключу параметров.

```
@CacheResult(cacheName = "prices")
public double price(String sku) {
    return expensiveLookup(sku);
}
```

> Проверено по руководству по кэшу: аннотация CacheResult с параметром cacheName существует и работает так, как в примере. Из руководства также следует, что при исключении в вызванном методе результат в кэш не попадает. Пример в рамке не менялся, он совпадает с формой из руководства.
>
> - К аргументу про ошибку под нагрузкой: в списке исправлений версии 3.40.1 есть запись о том, что CacheResult с Redis при высокой конкурентной нагрузке возвращал значение QUEUED. Источник
> - К аргументу про Redis-клиент и Dev Services: по руководству по Redis подключается расширение quarkus-redis-client, а в бин внедряются RedisDataSource для блокирующего стиля и ReactiveRedisDataSource для реактивного. Из источника данных берутся типизированные команды, например value или key, с методами get, set, incrby и del. Dev Services для Redis запускают сервер автоматически, это описано в материале Mastertheboss.

24. Планировщик и Quartz

- Встроенный планировщик задаёт периодические задачи аннотацией Scheduled с выражением cron или интервалом.
- Для кластерных сценариев с хранением состояния в базе подключается расширение Quartz.
- Задачи могут выполняться на виртуальных потоках или в реактивном контексте.
- Выражения можно брать из конфигурации и менять без пересборки.

Примеры к пункту 24

- К аргументу про аннотацию Scheduled: метод запускается по выражению cron или по интервалу.

```
@Scheduled(cron = "0 0 * * * ?")
void hourly() {
    // выполняется в начале каждого часа
}

@Scheduled(every = "30s")
void everyThirtySeconds() {
    // выполняется раз в 30 секунд
}
```

- К аргументу про выражения из конфигурации: значение берётся из свойства.

```
@Scheduled(cron = "{my.cron.expr}")
void configurable() { }
```

- Проверено по руководству по планировщику: параметры every и cron, а также запись выражения из конфигурации в фигурных скобках подтверждаются. Уточнение: по умолчанию выражения cron читаются в синтаксисе Quartz с секундами и знаком вопроса, а формат переключается свойством quarkus.scheduler.cron-type, поэтому шестиполевое выражение в моём примере написано верно. Название расширения quarkus-quartz для кластера я отдельно не проверяла.
- К аргументу про Quartz: для кластерных сценариев подключается отдельное расширение quarkus-quartz.

25. Camel Quarkus и другие интеграции

- Apache Camel доступен как набор расширений с оптимизацией под нативную сборку.
- Это открывает сотни коннекторов для типовых корпоративных паттернов интеграции.
- В экосистеме Quarkus более ста курируемых расширений, совместимых с нативной компиляцией.
- Совместимость с MicroProfile и Jakarta EE упрощает перенос существующего кода.

Примеры к пункту 25

- К аргументу про курируемые расширения: в сравнении Mad Devs сказано, что экосистема Quarkus включает более ста курируемых расширений. Источник
- К аргументу про совместимость со стандартами: там же отмечена совместимость с MicroProfile и Jakarta EE, а в описании на quarkus.io сказано про интеграцию со стандартами Jakarta и MicroProfile. Источник
- К аргументу про добавление расширения одной командой:

```
./mvnw quarkus:add-extension -Dextensions="camel-quarkus-core"
```

Проверено по документации Camel Quarkus: маршрут описывается классом, расширяющим RouteBuilder, и методом configure. Пример из документации выглядит так: from("timer:foo?period=1000").log("Hello World"). Для компонента timer нужно расширение camel-quarkus-timer, это следует из названия компонента, но точное имя артефакта я не видела в найденных страницах, поэтому найдите его в каталоге расширений.

- Примеров маршрутов Camel здесь нет: подтверждённых источников в моей подборке не было.

Группа 6. Облако, безопасность, наблюдаемость и тестирование

26. Kubernetes и контейнеры из коробки

- Расширения генерируют манифесты для Kubernetes и OpenShift прямо при сборке на основе конфигурации.
- Образы собираются через Jib, Docker или Podman без отдельных Dockerfile для простых случаев.
- Малый размер и быстрый старт делают проверки готовности и масштабирование более предсказуемыми.
- Новый тип упаковки для Maven в релизе 3.31 упрощает сборку.

> **Примеры к пункту 26**
>
> - К аргументу про генерацию манифестов при сборке: расширение Kubernetes создаёт манифесты по конфигурации проекта. Для локальной разработки есть расширение Minikube, которое создаёт файлы minikube.yaml и minikube.json. Источник

```
./mvnw quarkus:add-extension -Dextensions="quarkus-kubernetes"
./mvnw package
```

> Проверено: по руководству по Kubernetes после сборки в каталоге target/kubernetes появляются файлы kubernetes.json и kubernetes.yml, что подтверждает и статья Red Hat. Применить манифест можно командой kubectl apply -f target/kubernetes/kubernetes.yml (Quarkus Tutorial).
>
> - К аргументу про проверки готовности: расширение SmallRye Health реализует MicroProfile Health, а Spring Boot Actuator, как отмечено в статье Red Hat, даёт только конечные точки и не генерирует нужную конфигурацию и манифесты. Источник
> - К аргументу про образы: сборка контейнера включается свойствами.

```
quarkus.container-image.build=true
quarkus.container-image.name=my-service
quarkus.container-image.tag=latest
```

> Те же свойства использует пример нативной сборки в замере ForTek.

27. Безопасность: OIDC, JWT и OAuth 2.0

- Расширение OIDC поддерживает как роль защищённого сервиса, так и веб-приложения с входом через провайдера.
- В релизе 3.31 добавлено шифрование токенов OIDC и поддержка OAuth 2.0 Pushed Authorization Requests.
- Dev Services для OIDC поднимают провайдера, например Keycloak, на время разработки.
- Правила доступа задаются аннотациями вроде RolesAllowed и декларативной политикой в конфигурации.

> **Примеры к пункту 27**
>
> - К аргументу про Dev Services для OIDC: Dev Services для Keycloak включены по умолчанию, когда расширение quarkus-oidc работает в режиме разработки или тестов и свойство quarkus.oidc.auth-server-url не задано. Источник
> - К аргументу про готовность к работе сразу: контейнер Keycloak инициализируется с областью, клиентом и пользователями, и перезапускается при изменении application.properties или файла области. Источник
>
> ```
> <dependency>
>     <groupId>io.quarkus</groupId>
>     <artifactId>quarkus-oidc</artifactId>
> </dependency>
> ```
>
> - К аргументу про правила доступа аннотациями: метод закрывается проверкой роли.
>
> ```
> @GET
> @RolesAllowed("admin")
> public String secret() {
>     return "только для админов";
> }
> ```
>
> Проверено: аннотация RolesAllowed действительно используется для авторизации конечных точек, в том числе в руководстве по авторизации веб-конечных точек и в материале Baeldung. Эта аннотация стандартная, из Jakarta, так что мой пример сделан по принятой форме.
>
> - К аргументу про новое в релизе 3.31: добавлены шифрование токенов OIDC и поддержка OAuth 2.0 Pushed Authorization Requests. Источник

28. Метрики и трассировка: Micrometer и OpenTelemetry

- Расширения собирают метрики и трассы с минимальной настройкой, экспорт идёт в Prometheus или совместимые системы через OTLP.
- Автоматическая инструментация охватывает HTTP, клиенты баз данных и сообщения.
- Идентификаторы трассировки попадают в логи, что облегчает разбор инцидентов.
- Всё это работает и в нативных образах.

Примеры к пункту 28

- К аргументу про метрики отказоустойчивости: если подключено расширение Micrometer, SmallRye Fault Tolerance отправляет метрики в Micrometer, а иначе в метрики OpenTelemetry, если они доступны. Источник
- К аргументу про каталог руководств: в разделе Observability на quarkus.io есть отдельные записи для наблюдаемости и Dev Services. Источник
- К аргументу про подключение расширений:

```
./mvnw quarkus:add-extension -Dextensions="quarkus-opentelemetry,quarkus-micrometer-registry-prometheus"
```

Проверено: расширение OpenTelemetry называется quarkus-opentelemetry (руководство), а для метрик с выгрузкой в Prometheus используется quarkus-micrometer-registry-prometheus (каталог расширений). Команда выше использует именно эти полные имена. Сама форма команды add-extension в этом проходе не сверялась, её описывает руководство по Maven. Для OpenTelemetry в руководстве также показаны свойства: адрес экспортёра quarkus.otel.exporter.otlp.endpoint и формат логов с traceId, spanId и sampled, то есть идентификаторы трассировки в логах возможны.

- К аргументу про идентификаторы трассировки в логах: по руководству по OpenTelemetry они выводятся через формат консольного лога, например так.

```
quarkus.application.name=myservice
quarkus.otel.exporter.otlp.endpoint=http://localhost:4317
quarkus.log.console.format=%d{HH:mm:ss} %-5p traceId=%X{traceId}, parentId=%X{parentId}, spanId=%X{spanId}, sampled=%X{sampled} [%c{2.}] (%t) %s%e%n
```

Про нативные образы подтверждения в этом проходе не нашла, поэтому здесь про них ничего не утверждаю.

29. Проверки состояния и отказоустойчивость

- Расширение SmallRye Health даёт точки liveness, readiness и startup для оркестратора.
- SmallRye Fault Tolerance добавляет повторные попытки, таймауты, предохранитель и запасной вариант через аннотации.
- Правила настраиваются из конфигурации без изменения кода.

- Поведение хорошо сочетается с реактивными типами возврата.

> **Примеры к пункту 29**
>
> - К аргументу про включение отказоустойчивости: достаточно добавить расширение. Источник
>
> ```
> quarkus ext add io.quarkus:quarkus-smallrye-fault-tolerance
> ```
>
> - К аргументу про аннотации повторов, таймаутов и запасного варианта:
>
> ```
> @GET
> @Retry(maxRetries = 3, delay = 200)
> @Timeout(1000)
> @Fallback(fallbackMethod = "fallbackPrice")
> public double price() {
>     return remoteCall();
> }
>
> double fallbackPrice() {
>     return 0.0;
> }
> ```
>
> Проверено по руководству по Fault Tolerance и учебнику Quarkus: аннотации Retry с параметрами maxRetries и delay, Timeout и Fallback с параметром fallbackMethod существуют и используются так, как в моём примере. Уточнение: по умолчанию у Retry три повтора, а в руководстве таймаут задаётся в миллисекундах.
>
> - К аргументу про пробы для оркестратора: SmallRye Health реализует MicroProfile Health, то есть проверки liveness и readiness, которые использует Kubernetes. Источник
> - К аргументу про метрики: события отказоустойчивости уходят в Micrometer или OpenTelemetry, как описано выше. Источник

30. Тестирование: QuarkusTest, REST Assured и Testcontainers

- Аннотация QuarkusTest запускает приложение целиком, а Dev Services подставляют базу и брокеры.
- Релиз 3.31 перешёл на JUnit 6 и Testcontainers 2.
- Тесты можно запускать и против готового нативного образа, чтобы проверить итоговый артефакт.

- Для проверки виртуальных потоков есть отдельный фреймворк тестирования, важный на Java 25.

> **Примеры к пункту 30**
>
> - К аргументу про запуск приложения целиком в тесте:
>
> ```
> @QuarkusTest
> public class GreetingResourceTest {
>     @Test
>     public void testHelloEndpoint() {
>         given().when().get("/hello")
>             .then().statusCode(200);
>     }
> }
> ```
>
> - К аргументу про Dev Services в тестах: Dev Services для Keycloak работают и в режиме тестов, а контейнер базы данных для непрерывного тестирования запускается отдельно от режима разработки, чтобы тесты не мешали разработке. Источник
> - К аргументу про Testcontainers: руководство Testcontainers показывает проект с Panache, PostgreSQL и Flyway, где используется стиль Active Record. Источник
> - К аргументу про JUnit 6 и Testcontainers 2: переход произошёл в релизе 3.31. Источник
> - К аргументу про проверку виртуальных потоков: аннотация RunOnVirtualThread находится в пакете io.smallrye.common.annotation, а проверить, что код действительно на виртуальном потоке, можно вызовом Thread.currentThread().isVirtual(), как в разборе The Main Thread. Свойство quarkus.virtual-threads.name-prefix задаёт префикс имён таких потоков (Mastertheboss). К аргументу про нативный образ: по руководству по тестированию аннотация QuarkusIntegrationTest запускает артефакт сборки, будь то jar, нативный образ или контейнер. Старая аннотация NativeImageTest, по обсуждению в issue 18820, заменена на неё.

Источники и оговорки

- Quarkus 3.31: полная поддержка Java 25, Panache Next и другое
- Quarkus: главная страница проекта
- Quarkus: обсуждения поддержки JDK 25 на GitHub

- Часть технических деталей, например поведение отдельных расширений, описана по общим знаниям о проекте. Блоки примеров к пунктам с пометкой «Проверено» сверены с официальными руководствами Quarkus или с материалами, на которые стоят ссылки. Остальное проверяйте на quarkus.io перед внедрением.